

Meeting 03: The Empirical Reality Check — Reading Guide

Meeting 03: The Empirical Reality Check

Date: June 10, 2026

Contents

- Why This Meeting Exists
 - Agenda (60 Minutes)
 - The Lean Reading Bundle
 - Action Items
-

Why This Meeting Exists

Abstractions compress complexity. They do not eliminate it. This meeting pressure-tests John Ousterhout's design theories against concrete empirical data, hardware taxes, and organizational friction. Every reading and exercise has earned its place: zero academic filler, zero uncontextualized code blocks.

Total Reading: ~34 pages core, plus three 2-minute supplement reference cards.

Agenda (60 Minutes)

- **The Refresher (5 min):** Quick tactical recap of deep vs. shallow modules from Meeting 02. Refer to `02-essential-questions`.
 - **The Crucible (35 min):** Structured debate working through the three segments of the Lean Reading Bundle.
 - **The Application (15 min):** Interactive walkthrough of the **API Friction Checklist** and the **Vocabulary Audit** using the supplement cards.
 - **The Synthesis (5 min):** Final architectural conclusions and log logistics for Meeting 04.
-

The Lean Reading Bundle

1. Modern API Design & Information Hiding

- **Parnas (1972)** | *On the Criteria To Be Used in Decomposing Systems into Modules* | **6 pp**
 - [Read](#) — *The Classical Foundation*. A module's "secret" is the hard boundary between clean software evolution and system rot. Ousterhout's "deep modules" trace directly back to this paper.
- **Piccioni, Furia & Meyer (2013)** | *An Empirical Study of API Usability* | **10 pp**
 - [Sections I, II, IV](#) — Explores how interface complexity and poor type discovery mathematically drive up developer error rates.
 - *The Contemporary Foil*: Read this alongside [Topolog / Ivanov \(2025, Complexity Part 4: Abstractions\)](#). Topolog acts as a modern, high-density counterweight, extending Piccioni's interface metrics directly into modern UI components, SDK boundaries, and shallow cloud dependencies.
 - [Read alongside](#): The API Friction Checklist.
- **Stripe Engineering (2017)** | *APIs as infrastructure: future-proofing Stripe with versioning* | **6 pp**
 - [Read](#) — *Information Hiding at Scale*. A masterclass in change isolation using date-based rolling versions and encapsulated data transformations.
 - *The Contemporary Foil*: Read this alongside [Barroso \(Feb 2026, Abstraction: Designing Systems That Don't Collapse Under Complexity\)](#). Barroso provides an OOP-focused modern lens on how Stripe's real-world pipeline mechanics map to strict dependency inversion and volatility isolation.
 - [Read alongside](#): The Closed-Laptop Pattern (derived from Stripe's internal Lynx team mechanics).

2. Empirical Validation: Cognitive Load vs. Lint Rules

- **Bavota et al. (ICSE 2013)** | *An Empirical Study on Developers' Perception of Software Coupling* | **5 pp**
 - [Sections I, III, V](#) — Parametric formulas and effect sizes are stripped from the original paper. The raw empirical data is unmistakable: **semantic coupling governs human mental models far more than structural compilation links**. Structural coupling (what code calls what) matters less than semantic coupling (what developers name things). Shared vocabulary across supposedly decoupled services is the silent coupling that breaks production.
 - [Read alongside](#): Skim the **Temple Analogy** and **Vocabulary Audit**.

3. Runtime & Abstraction Tax

- **Lemire (2022)** | *Avoid exception throwing in performance-sensitive code* | **3 pp**
 - [Read](#) — *The Execution Tax*. When an elegant syntactic abstraction leaks across a module boundary, the resulting $10,000\times$ performance drop is the physical proof of encapsulation breaking down at the CPU level.
 - [Read alongside](#): Runtime Tax supplement.

- **Java Javadoc Core** | *Package Summary: java.nio & java.nio.channels.FileChannel* | **4 pp**
 - Bypasses the broken tutorial links. Read the package-level description of how *Buffers* decouple memory from execution channels. Focus specifically on **Direct vs. Non-Direct Buffers**: the exact structural moment where an elegant abstraction layer is forced to reveal its underlying native hardware reality to protect system performance.
 - [java.nio Package Specification](#)
 - [FileChannel Documentation \(Direct vs. Non-Direct Buffers\)](#)
 - [Read alongside](#): Runtime Tax supplement.
-

Action Items

- [] Read the Lean Reading Bundle (~31 pp core total)
- [] Review the three targeted supplement reference cards in this directory
- [] Execute: Run the **Vocabulary Audit** on an internal repository before June 10, and bring your identified hidden ley lines to the group session